NumPy: Array Manipulation, Statistical Functions, and Random Sampling

These topics are essential in **Data Science** and **Machine Learning** because they help you reshape datasets, analyze data, and generate random values for testing and model training.

import numpy as np

Part A: Array Manipulation

Array manipulation means **changing the shape, size, or arrangement of an array** without changing its data.

---

1. Reshape

Changes the shape of an array.

import numpy as np

arr = np.arange(12)

print(arr)

Output

[0 1 2 3 4 5 6 7 8 9 10 11]

Reshape into 3 rows and 4 columns.

new_arr = arr.reshape(3,4)

(new_arr)

Output

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

---

2. Flatten

Converts a multi-dimensional array into a 1D array.

```
arr = np.array([[1,2],
                [3,4]])

print(arr.flatten())
```

Output

```
[1 2 3 4]
```

---

3. Ravel

Similar to `flatten()`, but returns a **view** when possible.

```
arr = np.array([[1,2],
                [3,4]])

print(arr.ravel())
```

Output

```
[1 2 3 4]
```

Difference

- `flatten()` → creates a copy.

- `ravel()` → returns a view if possible (more memory efficient).

---

4. Transpose

Interchanges rows and columns.

```
arr = np.array([[1,2,3],
                [4,5,6]])

print(arr.T)
```

Output

```
[[1 4]
 [2 5]
 [3 6]]
```

---

5. Resize

Changes the size of an array.

```
arr = np.array([1,2,3,4])

print(np.resize(arr,(2,3)))
```

Output

```
[[1 2 3]
 [4 1 2]]
```

If more elements are needed, NumPy repeats them.

---

6. Concatenate

Joins arrays.

```
a = np.array([1,2,3])

b = np.array([4,5,6])

print(np.concatenate((a,b)))
```

Output

```
[1 2 3 4 5 6]
```

---

7. Vertical Stack

```
a = np.array([1,2,3])

b = np.array([4,5,6])

print(np.vstack((a,b)))
```

Output

```
[[1 2 3]
 [4 5 6]]
```

---

8. Horizontal Stack

```
print(np.hstack((a,b)))
```

Output

```
[1 2 3 4 5 6]
```

---

9. Column Stack

print(np.column_stack((a,b)))

Output

```
[[1 4]
 [2 5]
 [3 6]]
```

---

10. Split

Splits arrays into equal parts.

arr = np.array([1,2,3,4,5,6])

print(np.split(arr,3))

Output

```
[array([1,2]),
 array([3,4]),
 array([5,6])]
```

---

11. Append

arr = np.array([1,2,3])

print(np.append(arr,4))

Output

[1 2 3 4]

---

12. Insert

arr = np.array([1,2,4])

print(np.insert(arr,2,3))

Output

[1 2 3 4]

---

13. Delete

arr = np.array([1,2,3,4])

print(np.delete(arr,1))

Output

[1 3 4]

Part B: Statistical Functions

These functions summarize and analyze data.

Suppose

arr = np.array([10,20,30,40,50])

Mean (Average)

```
print(np.mean(arr))
```

Output

```
30.0
```

Median

```
print(np.median(arr))
```

Output

```
30.0
```

Standard Deviation

Measures the spread of the data.

```
print(np.std(arr))
```

Output

```
14.142135623730951
```

Variance

Square of the standard deviation.

```
print(np.var(arr))
```

Output

```
200.0
```

Minimum

print(np.min(arr))

Output

10

Maximum

print(np.max(arr))

Output

50

Sum

print(np.sum(arr))

Output

150

Product

print(np.prod(arr))

Output

12000000

Cumulative Sum

```
print(np.cumsum(arr))
```

Output

```
[ 10  30  60 100 150]
```

Cumulative Product

```
print(np.cumprod(arr))
```

Output

```
[      10      200     6000   240000 12000000]
```

Percentiles

```
print(np.percentile(arr,25))
print(np.percentile(arr,50))
print(np.percentile(arr,75))
```

Output

```
20.0
30.0
40.0
```

Average (Weighted)

```
marks = np.array([60,70,80])

weights = np.array([1,2,3])

print(np.average(marks,weights=weights))
```

Output

73.33333333333333

Part C: Random Sampling

Random numbers are widely used in Machine Learning for:

- Initializing model weights
- Splitting datasets
- Data augmentation
- Simulations
- Testing algorithms

---

1. Random Integer

print(np.random.randint(1,10,size=5))

Example Output

[7 2 8 5 1]

---

2. Random Float

print(np.random.rand(5))

Example Output

[0.34 0.72 0.11 0.95 0.28]

---

3. Random Array with Shape

```
print(np.random.rand(3,2))
```

Example Output

```
[[0.62 0.21]
 [0.87 0.19]
 [0.44 0.78]]
```

---

4. Normal Distribution

Random values from a normal (Gaussian) distribution.

```
print(np.random.randn(5))
```

Example Output

```
[-0.51  1.32 -0.27  0.86 -1.44]
```

---

5. Random Choice

```
colors = ["Red","Blue","Green"]

print(np.random.choice(colors))
```

Example Output

```
Green
```

---

6. Random Choice with Probabilities

```
colors = ["Red","Blue","Green"]

print(np.random.choice(colors, p=[0.2,0.3,0.5]))
```

---

7. Shuffle

Randomly rearranges an array.

```
arr = np.arange(10)

np.random.shuffle(arr)

print(arr)
```

Example Output

```
[5 8 1 0 3 9 2 4 6 7]
```

---

8. Permutation

Returns a shuffled copy without changing the original array.

```
arr = np.arange(5)

print(np.random.permutation(arr))
```

---

9. Seed

Generates the same random values every time the program runs.

```
np.random.seed(42)

print(np.random.randint(1,100,5))
```

Output

[52 93 15 72 61]

This is useful for reproducible machine learning experiments.

Quick Reference Table

Function	Purpose
`reshape()`	Change array shape
`flatten()`	Convert to 1D (copy)
`ravel()`	Convert to 1D (view if possible)
`transpose()` / `.T`	Swap rows and columns
`resize()`	Resize array
`concatenate()`	Join arrays
`vstack()`	Stack vertically
`hstack()`	Stack horizontally
`column_stack()`	Combine arrays as columns
`split()`	Split an array
`append()`	Add element(s)
`insert()`	Insert element(s)
`delete()`	Remove element(s)
`mean()`	Average
`median()`	Middle value

`std()`	Standard deviation
`var()`	Variance
`min()` / `max()`	Smallest / largest value
`sum()` / `prod()`	Sum / product of elements
`cumsum()` / `cumprod()`	Cumulative sum / product
`percentile()`	Percentile values
`average()`	Weighted average
`randint()`	Random integers
`rand()`	Uniform random floats
`randn()`	Normally distributed values
`choice()`	Random selection
`shuffle()`	Shuffle in place
`permutation()`	Return shuffled copy
`seed()`	Reproducible random numbers

Pandas Complete Notes

What is Pandas?

Pandas is an open-source Python library used for:

- Reading datasets
- Cleaning data
- Transforming data
- Analyzing data
- Preparing data for Machine Learning

Almost every Data Science or Machine Learning project starts with Pandas.

import pandas as pd

Core Data Structures

Pandas has only two primary data structures.

Pandas

├── Series (1D)

└── DataFrame (2D)

---

1. Series

A Series is a one-dimensional labeled array.

Think of it as a single column in Excel.

Example

import pandas as pd

ages = pd.Series([18, 20, 22, 25])

print(ages)

Output

0 18

1 20

2 22

3 25

dtype: int64

Every Series has:

- values
- index

Index Value

0 --------> 18

1 --------> 20

2 --------> 22

3 --------> 25

Custom Index

```
ages = pd.Series(
   [18,20,22],
   index=["Rahul","Anu","John"]
)
```

```
print(ages)
```

Output

```
Rahul    18
Anu      20
John     22
```

Access

```
print(ages["Rahul"])
```

Output

```
18
```

---

2. DataFrame

The most important object.

Think of it as an Excel spreadsheet.

```
     Name     Age     Marks

0    John     20      95

1    Anna     22      90
```

2 Mike 19 87

Create manually

```
data = {
    "Name":["John","Anna","Mike"],
    "Age":[20,22,19],
    "Marks":[95,90,87]
}

df = pd.DataFrame(data)

print(df)
```

DataFrame Anatomy

Columns

Name Age Marks

Index

0 John 20 95

1 Anna 22 90

2 Mike 19 87

A DataFrame contains:

- Rows
- Columns
- Index
- Values

Reading Data

CSV

df = pd.read_csv("students.csv")

Excel

df = pd.read_excel("students.xlsx")

JSON

df = pd.read_json("students.json")

Saving Data

df.to_csv("new.csv")

Without index

df.to_csv("new.csv", index=False)

Inspecting Data

head()

First rows

df.head()

First 5 rows

Custom

df.head(10)

---

tail()

df.tail()

Last rows.

---

sample()

Random rows.

df.sample(3)

---

shape

Attribute

df.shape

Output

(1000,5)

Means

1000 rows

5 columns

---

columns

df.columns

Output

Index(['Name','Age','Salary'])

---

dtypes

df.dtypes

Output

Name object

Age int64

Salary float64

---

info()

One of the most useful functions.

df.info()

Shows

- rows
- columns
- missing values
- datatype
- memory usage

---

describe()

df.describe()

Returns statistics

count

mean

std

min

25%

50%

75%

max

Very useful for numeric data.

Selecting Data

Suppose

Name Age Marks

John 20 90

Anna 21 80

Mike 19 70

Single Column

df["Name"]

Returns Series.

Multiple Columns

df[["Name","Marks"]]

Returns DataFrame.

Selecting Rows

Using loc

df.loc[0]

Returns first row.

Multiple rows

df.loc[0:2]

Specific row and column

df.loc[0,"Marks"]

Output

90

---

iloc

Uses positions.

df.iloc[0]

First row

df.iloc[0,2]

First row

Third column

Difference

loc

Labels

iloc

Positions

Filtering Data

Students older than 20

df[df["Age"]>20]

Marks greater than 80

df[df["Marks"]>80]

Multiple Conditions

AND

df[(df["Age"]>20)&(df["Marks"]>80)]

OR

df[(df["Age"]>20)|(df["Marks"]>80)]

---

isin()

```
df[df["City"].isin(["Delhi","Mumbai"])]
```

---

between()

```
df[df["Age"].between(20,30)]
```

Adding Columns

```
df["Passed"]=True
```

From another column

```
df["Bonus"]=df["Salary"]*0.10
```

Renaming Columns

```
df.rename(columns={
    "Marks":"Score"
})
```

Removing Columns

df.drop(columns=["Age"])

Removing rows

df.drop(index=0)

Missing Values

NaN

Means data missing.

Check

df.isnull()

Count

df.isnull().sum()

Remove

df.dropna()

Fill

df.fillna(0)

Fill with average

```
df["Age"]=df["Age"].fillna(df["Age"].mean())
```

Duplicate Values

Check

```
df.duplicated()
```

Remove

```
df.drop_duplicates()
```

Sorting

Ascending

```
df.sort_values("Marks")
```

Descending

```
df.sort_values(
    "Marks",
    ascending=False
)
```

Statistics

df["Marks"].mean()

df["Marks"].median()

df["Marks"].mode()

df["Marks"].std()

df["Marks"].sum()

df["Marks"].min()

df["Marks"].max()

---

value_counts()

Frequency

df["City"].value_counts()

Output

Delhi 10

Mumbai 8

Kochi 4

Unique Values

df["City"].unique()

Count

df["City"].nunique()

GroupBy (One of the Most Important Topics)

Suppose:

Department Salary

IT 50000

HR 30000

IT 60000

HR 40000

Average salary by department

df.groupby("Department")["Salary"].mean()

Output

HR 35000

IT 55000

Multiple calculations

df.groupby("Department")["Salary"].agg(

    ["mean","max","min","count"]

)

Merge (SQL-style Join)

Employee table

ID Name

1 John

2 Anna

Salary table

ID Salary

1 50000

2 60000

```
pd.merge(
    employee,
    salary,
    on="ID"
)
```

Result

ID Name Salary

1 John 50000

2 Anna 60000

Concatenate

pd.concat([df1,df2])

Stacks DataFrames together.

String Operations

Convert to lowercase

df["Name"].str.lower()

Uppercase

df["Name"].str.upper()

Contains

df["Email"].str.contains("@gmail")

Replace

df["Name"].str.replace("John","Jack")

Length

df["Name"].str.len()

Datetime

Convert

df["Date"]=pd.to_datetime(df["Date"])

Extract

df["Date"].dt.year

df["Date"].dt.month

df["Date"].dt.day

df["Date"].dt.day_name()

Apply

Apply a function to every value in a column.

df["Marks"].apply(lambda x:x+5)

Map

Replace values.

gender={

```
    "M":"Male",
    "F":"Female"
}


df["Gender"]=df["Gender"].map(gender)
```

Common Workflow in a Real ML Project

1. Import pandas.
2. Load the dataset (`read_csv()`).
3. Inspect it (`head()`, `info()`, `describe()`).
4. Check and handle missing values (`isnull()`, `fillna()`, `dropna()`).
5. Remove duplicates (`drop_duplicates()`).
6. Filter or select relevant rows and columns.
7. Create new features (new columns).
8. Group or summarize data (`groupby()`).
9. Merge datasets if needed (`merge()`).
10. Save the cleaned data (`to_csv()`) or pass it to a machine learning model.

High-Priority Pandas Topics for AI/ML

Priority	Topics
⭐⭐⭐⭐⭐	DataFrame, Series, `read_csv()`, `head()`, `info()`, `describe()`
⭐⭐⭐⭐⭐	Column selection, `loc`, `iloc`, filtering

⭐⭐⭐⭐⭐	Missing values (`isnull()`, `fillna()`, `dropna()`)
⭐⭐⭐⭐⭐	`groupby()`, `agg()`
⭐⭐⭐⭐	`sort_values()`, `value_counts()`, `unique()`
⭐⭐⭐⭐	`merge()`, `concat()`
⭐⭐⭐⭐	String methods (`.str`) and datetime methods (`.dt`)
⭐⭐⭐	`apply()`, `map()`, `rename()`, `drop()`

These three topics cover almost everything you need to know in **Matplotlib** for Machine Learning and Data Science. Here's a structured syllabus with explanations, important functions, and examples.

---

a) Basic Plotting, Customization, and Subplots

1. Introduction to Matplotlib

- What is Matplotlib?
- Why use Matplotlib?
- Installing and importing
- `matplotlib.pyplot` (`plt`)
- Basic plotting workflow

import matplotlib.pyplot as plt

---

2. Basic Plotting

`plt.plot()`

Creates a line graph.

```
import matplotlib.pyplot as plt


x = [1,2,3,4,5]

y = [10,20,15,25,30]


plt.plot(x, y)

plt.show()
```

Important Parameters

Parameter	Description
`x`	X-axis values
`y`	Y-axis values
`label`	Legend name

`color`	Line color
`linewidth`	Thickness
`linestyle`	Style of line
`marker`	Point marker

Example

```
plt.plot(
    x,
    y,
    color="red",
    linewidth=2,
    linestyle="--",
    marker="o",
    label="Sales"
)
```

---

3. Figure Creation

`plt.figure()`

Creates a new figure.

plt.figure(figsize=(8,5))

Parameters

- figsize
- dpi
- facecolor

---

4. Labels

Title

plt.title("Monthly Sales")

X-axis Label

plt.xlabel("Month")

Y-axis Label

plt.ylabel("Revenue")

---

5. Legend

plt.legend()

Example

plt.plot(x,y,label="2025")

```
plt.legend()
```

---

6. Grid

```
plt.grid(True)
```

---

7. Axis Limits

```
plt.xlim(0,10)


plt.ylim(0,100)
```

---

8. Axis Ticks

```
plt.xticks([1,2,3,4])


plt.yticks([0,20,40,60])
```

---

9. Saving Figures

```
plt.savefig("sales.png")
```

---

10. Showing Plot

plt.show()

Subplots

Subplots allow multiple graphs in one figure.

`plt.subplot()`

Syntax

plt.subplot(rows, columns, position)

Example

plt.subplot(1,2,1)

plt.plot(x,y)

plt.subplot(1,2,2)

plt.bar(x,y)

plt.show()

---

plt.subplots()

Preferred method.

fig, ax = plt.subplots(2,2)

Example

fig, ax = plt.subplots(2,2)

ax[0,0].plot(x,y)

ax[0,1].scatter(x,y)

ax[1,0].bar(x,y)

ax[1,1].hist(y)

plt.show()

---

tight_layout()

Automatically fixes spacing.

plt.tight_layout()

---

b) Histograms, Bar Charts, Scatter Plots

---

1. Histogram

Purpose

Shows data distribution.

Example

```
marks = [50,60,70,75,80,85,90,90,95]


plt.hist(marks)

plt.show()
```

Important Parameters

Parameter	Purpose
bins	Number of intervals
edgecolor	Border color
density	Probability density
alpha	Transparency

Example

```
plt.hist(
    marks,
    bins=5,
```

```
    edgecolor="black"
)
```

Use Cases

- Age distribution
- Salary distribution
- Marks distribution

---

2. Bar Chart

Purpose

Compare categories.

Example

```
students = ["A","B","C"]

marks = [90,80,95]

plt.bar(students, marks)
plt.show()
```

Horizontal Bar

```
plt.barh(students, marks)
```

Parameters

Parameter	Purpose
width	Bar width
color	Bar color
edgecolor	Border
align	Alignment

Use Cases

- Sales comparison
- Student marks
- Population

---

3. Scatter Plot

Purpose

Shows relationship between variables.

Example

height = [160,165,170,175]

weight = [55,60,70,80]

```
plt.scatter(height, weight)

plt.show()
```

Parameters

Parameter	Purpose
s	Marker size
c	Marker color
marker	Marker shape
alpha	Transparency

Use Cases

- Height vs Weight
- Study Hours vs Marks
- Experience vs Salary

---

c) Additional Features and Use Cases

---

1. Pie Chart

```
labels = ["Python","Java","C++"]


sizes = [50,30,20]


plt.pie(
    sizes,
    labels=labels,
    autopct="%1.1f%%"
)


plt.show()
```

Use

- Market share
- Budget allocation

---

2. Box Plot

```
salary = [20,25,30,40,90]


plt.boxplot(salary)


plt.show()
```

Use

- Detect outliers
- Compare distributions

---

3. Multiple Lines

```
plt.plot(x,y1,label="2024")


plt.plot(x,y2,label="2025")


plt.legend()


plt.show()
```

---

4. Annotations

```
plt.annotate(
    "Highest",
    xy=(5,30)
)
```

---

5. Text

```
plt.text(
```

```
    2,
    20,
    "Important"
)
```

---

6. Style Sheets

```
plt.style.use("ggplot")
```

Popular styles

- ggplot
- classic
- dark_background
- fast

---

9. Working with Pandas

```
import pandas as pd

df = pd.read_csv("students.csv")

plt.scatter(
    df["Height"],
    df["Weight"]
)
```

```
plt.show()
```

---

10. Working with NumPy

```
import numpy as np


x = np.linspace(0,10,100)


y = np.sin(x)


plt.plot(x,y)


plt.show()
```

Commonly Used Functions

Plotting

plt.plot()

plt.scatter()

plt.bar()

plt.barh()

plt.hist()

plt.pie()

plt.boxplot()

Customization

plt.title()

plt.xlabel()

plt.ylabel()

plt.legend()

plt.grid()

plt.xlim()

plt.ylim()

plt.xticks()

plt.yticks()

Figure Management

plt.figure()

plt.subplot()

plt.subplots()

plt.tight_layout()

Saving

plt.savefig()

plt.show()

plt.close()

Real-World Data Science Workflow

1. Load the dataset using **Pandas** (`pd.read_csv()`).
2. Inspect the data (`head()`, `info()`, `describe()`).
3. Create visualizations:
    - **Histogram** → Check data distribution.
    - **Scatter Plot** → Explore relationships between features.
    - **Bar Chart** → Compare categories.
    - **Box Plot** → Detect outliers.
4. Customize plots with titles, labels, legends, and grids.
5. Arrange multiple plots using `subplots()`.
6. Save figures for reports using `savefig()`.

Some Important Questions

Topic Pending and Need improvement

Selecting Data

- Single column
- Multiple columns
- `.loc`
- `.iloc`

Filtering Data ← Your Question #3

- `>`
- `<`
- `==`
- `!=`
- `&`
- `|`
- `.isin()`
- `.between()`

Column Operations ← Your Question #1

- Add columns
- Rename columns
- Delete columns
- Update values
- `apply()`

Row Operations

- Add rows
- Delete rows
- Update rows

The Pending from the review

1. ***Add a column to Pandas DataFrame***
2. ***Find total number of rows and columns***
3. ***Filtering a DataFrame based on a condition***

Priority for AI/ML

Priority	Topics
⭐⭐⭐⭐⭐	`plot()`, `scatter()`, `hist()`, `bar()`
⭐⭐⭐⭐⭐	`title()`, `xlabel()`, `ylabel()`, `legend()`, `grid()`
⭐⭐⭐⭐⭐	`figure()`, `subplots()`, `show()`, `savefig()`
⭐⭐⭐⭐	`boxplot()`, `pie()`

⭐⭐⭐⭐ Axis limits, ticks, markers, colors, line styles